

3.1.4 Dictionaries (Key-Value pairs)

ڈکشنری (Dictionary) کلید اور قدر (Key-Value) کے جوڑوں میں ڈیٹا رکھتی ہے۔

CS XII

Chapter 3: Programming Fundamentals

Lecture 05 of 17

★ Golden

3.1.4 Dictionaries (Key-Value pairs) ★ Golden

LEARN IT

A **dictionary** stores data as **key-value pairs**. It works like a real dictionary: you look up a word (the **key**) to find its meaning (the **value**). Write a dictionary with curly braces, and put a **colon** between each key and value.

- **Ordered** (Python 3.7+): insertion order is kept.
- **Mutable**: you can add, change and delete pairs.
- **Keys are unique**: if you repeat a key, the last value silently overwrites the earlier one. Values may repeat.



You never search a dictionary by position. You search by key.

A school analogue: a **roll number** (key) can point to only one student (value). If you assign that roll number to someone else, the old link is gone.

Adding, changing and deleting

```
d = {"name": "Alice", "age": 30}
print(d)
d["age"] = 31
print(d)
d["city"] = "NYC"
print(d)
del d["city"]
print(d)
```

OUTPUT

```
{'name': 'Alice', 'age': 30}
{'name': 'Alice', 'age': 31}
{'name': 'Alice', 'age': 31, 'city': 'NYC'}
{'name': 'Alice', 'age': 31}
```

Assigning to an existing key updates the value. Assigning to a new key adds a pair. `del d["city"]` removes that pair.

The duplicate-keys rule. Python does not crash. The last assignment wins.

```
d = {"a": 1, "a": 2, "a": 3}
print(d)
```

```
OUTPUT
{'a': 3}
```

Accessing values safely

1. Direct brackets (risky)

`dict[key]` works if the key exists. If it is missing, the program crashes with `KeyError`.

2. `.get()` (safe)

`dict.get(key)` returns `None` if the key is missing. You may pass a default:

```
dict.get(key, 0).
```

```
person = {"name": "Alice", "age": 30}
print(person["name"])
print(person.get("salary"))
print(person.get("salary", 0))
print("name" in person)
print("salary" in person)
```

```
OUTPUT
Alice
None
0
True
False
```

The operator `in` tests **keys**, not values. `"Alice" in person` would be `False`.

```
person = {"name": "Alice", "age": 30}
print(person["salary"])
```

```
OUTPUT
Traceback (most recent call last):
  File "keyerror.py", line 2, in <module>
    print(person["salary"])
KeyError: 'salary'
```

Dictionary views and traversal

Three view methods return live objects. If the dictionary changes, the view reflects the change immediately.

Method	Returns
--------	---------

<code>.keys()</code>	All keys.
<code>.values()</code>	All values.
<code>.items()</code>	Key-value pairs as tuples.

```
d = {"a": 1}
keys = d.keys()
print(keys)
d["b"] = 2
print(keys)
```

OUTPUT

```
dict_keys(['a'])
dict_keys(['a', 'b'])
```

The variable `keys` was not rebuilt. The view updated itself.

```
inventory = {"apple": 50, "banana": 30, "cherry": 20}
print(list(inventory.keys()))
print(list(inventory.values()))
print(list(inventory.items()))
for key in inventory:
    print(key)
for val in inventory.values():
    print(val)
for key, val in inventory.items():
    print(f"{key} costs {val}")
```

OUTPUT

```
['apple', 'banana', 'cherry']
[50, 30, 20]
[('apple', 50), ('banana', 30), ('cherry', 20)]
apple
banana
cherry
50
30
20
apple costs 50
banana costs 30
cherry costs 20
```

A plain `for key in inventory` loops over keys (the default). `.items()` is the most useful form because you get both sides at once.

Comparison: list, tuple, set, dictionary

Feature	List	Tuple	Set	Dictionary
---------	------	-------	-----	------------

Written with	<code>[]</code>	<code>()</code>	<code>{ }</code>	<code>{key: value}</code>
Ordered	Yes	Yes	No	Yes (Python 3.7+)
Mutable	Yes	No	Yes (items must be immutable)	Yes
Duplicates	Allowed	Allowed	Removed	Duplicate keys not allowed (last value wins)
Access by	Index 0, 1, 2...	Index 0, 1, 2...	No index	Key
Empty value	<code>[]</code>	<code>()</code>	<code>set()</code>	<code>{}</code>
Typical use	Changing sequences	Fixed records	Unique items, set maths	Look-up tables

WORKED EXAMPLE

Same data, four structures. Three subject marks:

- List `[80, 70, 90]` — easy to append a fourth test.
- Tuple `(80, 70, 90)` — a frozen result slip.
- Set `{80, 70, 90}` — unique scores only; two papers of 80 would become one 80.
- Dictionary `{"math": 80, "eng": 70, "cs": 90}` — you ask "What is CS?" and get 90 without remembering an index.

REMEMBER IT

Keys are unique, like roll numbers. Values are the student records and may repeat (two students can both score 80). Memory: "**get** is gentle, brackets can **break**."

COMMON MISTAKE

Older notes (and the chapter summary in some slides) still call dictionaries "unordered." From Python 3.7 onwards insertion order is preserved, so you may write "ordered since 3.7." Keys must be immutable: a list cannot be a key; a tuple of numbers can. Do not write `{}` when you want an empty set.

ڈکشنری (Dictionary) کلید اور قدر (Key-Value) کے جوڑوں میں ڈیٹا رکھتی ہے۔ کلید منفرد ہوتی ہے؛ دہرائی ہوئی کلید پر آخری قدر رہ جاتی ہے۔ `d[key]` غیر موجود کلید پر `KeyError` دیتا ہے، جبکہ `d.get(key, default)` محفوظ ہے۔ `.values()`، `keys()` اور `items()` سے گھما جا سکتا ہے۔ فہرست انڈیکس سے، ڈکشنری کلید سے پہچانی جاتی ہے۔

EXAM TIP

High-yield: definition of a dictionary, difference between `d[key]` and `d.get(key)`, "what if a key is repeated?", traversal with `.items()`, and a full comparison table of the four structures. Memorise the comparison table in this section — it is a complete 8-mark answer if you add one example row. Always mention that `in` checks keys.

CHECK YOURSELF

1. What does `{"a": 1, "a": 2, "a": 3}` print?
2. What does `person.get("salary", 0)` return if `salary` is missing?
3. Which structure has no index and no duplicate items?

Answers: 1. `{'a': 3}`. 2. `0`. 3. A set.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. A dict stores:

- a. key: value pairs
- b. Only a list of values
- c. Only unique numbers without keys
- d. Only files

MCQ 2. Keys in a dict must be:

- a. Hashable (immutable) and unique
- b. Lists
- c. Sets
- d. Other dicts as keys commonly

MCQ 3. `d.get("x")` if missing:

- a. Returns None (or default) instead of crashing
- b. Always `KeyError`
- c. Deletes `d`
- d. Sorts `d`

MCQ 4. `d["x"]` if missing:

- a. `KeyError`
- b. None always
- c. 0 always
- d. Inserts `NAND`

MCQ 5. `d.keys()` is:

- a. A view of keys
- b. A tuple of values
- c. A file
- d. A stack

MCQ 6. Updating `d["a"]=2` if `a` exists:

- a. Overwrites
- b. Creates a second `a`
- c. Errors always
- d. Appends a list

Extra short questions

1. Define dictionary with a 2-pair example.
2. Safe read vs direct read.
3. How to loop keys and values.
4. Why lists cannot be keys.

Extra long question

1. Write a marks dict program: add a student, update, print using `items()`, handle a missing name with `get`. Sample I/O.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. {"Ali": 85, "Sara": 90}
2. get vs [].
3. for k,v in d.items():
4. Mutable, unhashable.

Long

1. CRUD-ish four operations. KeyError demo vs get. 10-mark complete code.

Next lecture: 3.1.5 Common Built-in Functions (min, max, len, sum)